Acciones Procedurales en Bases de Datos

Triggers, funciones y stored procedures

Ejercicio 1

a) Para el esquema de *Proveedores*, implemente mediante triggers en PostgreSQL los controles de los ejercicios 3a) y 3d) del Práctico 2. Previamente complete una tabla como la siguiente con los eventos críticos a controlar en cada caso.

A modo de guía, dada la siguiente solución declarativa para el chequeo: *Cada proveedor sólo puede proveer productos a sucursales de su localidad*

```
create assertion chk_localidades check not exists (
 select 1 from proveedor p
      join provee pr on (pr.nro_prov = p.nro_prov)
      join sucursal s on (s.cod_suc = pr.cod_suc)
      where p.localidad <> s.localidad );
```

se deberían analizar cuáles son los eventos críticos que se requieren controlar y completar la tabla a continuación con los datos necesarios para completar la cabecera de el/los trigger/s requerido/s:

Tabla / Evento	*Insert*	*Update*	*Delete*	*Granularidad*
PROVEEDOR	no / si	no / si (atributo/s)	no / si	f. e. row / statement
PROVEE	no / si	no / si (atributo/s)	no / si	f. e. row / statement
SUCURSAL	no / si	no / si (atributo/s)	no / si	f. e. row / statement

b) Provea al menos 2 sentencias disparadoras para el/los trigger/s definidos (sobre distintas tablas y/o eventos) que provoquen su activación, ejecútelas sobre la base y chequee la respuesta del DBMS.

Ejercicio 2

Para las restricciones solicitadas en el ejercicio 4 del Práctico 2, determine los eventos críticos (puede orientarse mediante la tabla anterior) e implemente los triggers para aquellos chequeos que no sea posible incorporar declarativamente en PostgreSQL.

Ejercicio 3

Considere dos tablas EMPLEADO_1 y EMPLEADO_2 con atributos (id_empleado, nombre, apellido, sueldo) y que contienen las siguientes tuplas (*Nota: se indican los valores de id_empleado y sueldo respectivamente*) :

EMPLEADO_1: <1,....,500> y EMPLEADO_2: <2,...,700>, <3,....,300>, <4,....,700>

Suponga que se define el siguiente trigger en PostgreSQL:

```
CREATE OR REPLACE FUNCTION autoIncremento()
RETURNS TRIGGER AS $$
BEGIN
  UPDATE empleado_1 SET
     sueldo = sueldo + (SELECT min(sueldo)*0.05 FROM empleado_1);
  RETURN [ … ];
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_autoincremento
BEFORE INSERT ON Empleado_1
[ granularidad ]
EXECUTE FUNCTION autoIncremento();
```

y que luego, por medio de la sentencia INSERT INTO EMPLEADO_1 SELECT * FROM EMPLEADO_2; se insertan en EMPLEADO_1 las tuplas que contiene EMPLEADO_2. Indique el estado final de la tabla EMPLEADO_1 si: a) [granularidad]= for each row ; b) [granularidad]= for each statement.

Acciones Procedurales en Bases de Datos

Triggers, funciones y stored procedures

Ejercicio 4

En el esquema de Películas (unc_esq_peliculas), considere que se quiere mantener un registro de quién y cuándo realizó actualizaciones sobre las entregas de películas. Cree una tabla HIS_ENTREGA que tenga por lo menos las siguientes columnas: nro_registro, fecha, operación, cant_reg_afectados, usuario.

a) Provea el/los trigger/s necesario/s para mantener actualizada en forma automática la tabla HIS_ENTREGA cuando se realizan actualizaciones (insert, update o delete) en la tabla ENTREGA o RENGLON_ ENTREGA

b) Determine el resultado en las tablas si se ejecuta la operación:
DELETE FROM ENTREGA WHERE id_video = 3582 ;

según el o los triggers definidos sean FOR EACH ROW o FOR EACH STATEMENT, evalúe la diferencia a partir de ambos tipos de granularidad.

Nota: considere que la RIR fk_re_entrega definida entre RENGLON_ENTREGA y ENTREGA posee modalidad cascade para update y delete.

Ejercicio 5

En el esquema de Películas, se desea llevar otra tabla histórica en la cual conste, para cada EMPLEADO, el tiempo que ha permanecido en el sistema y el tiempo promedio en cada departamento.

Plantee los cambios necesarios en el esquema para contemplar ambos datos (mediante la/s sentencia/s SQL necesaria/s) y luego:

- Plantee un stored procedure que realice la inicialización de esta tabla (considere que recibe por parámetro la fecha en la cual entraron todos los empleados).
- Implemente esta actualización mediante trigger/s.
- ¿Un procedimiento es posible que mantenga actualizada en todo momento la información? ¿Por qué?
- ¿Qué ocurriría con la información pre-existente en la base de datos al momento de incorporar el o los triggers; y el procedimiento?
- Analice si se podría implementar lo anterior mediante chequeos declarativos (de tabla o generales).

Ejercicio 6

Para el esquema Voluntarios, implemente un trigger que:

a) Ante un cambio de tarea de un voluntario, coloque en 0 la cantidad de horas aportadas, independientemente de si el valor de horas_aportadas de la sentencia update es diferente.

b) Cuando se actualice la cantidad de horas aportadas, el nuevo valor no puede ser ni menor, ni superior al 10 % del valor que tenía anteriormente.

Ejercicio 7

Para el esquema del ejercicio 2 del Práctico 2, cree la siguiente tabla en la que se requiere llevar registro de la cantidad de textos publicados y la fecha de la última publicación, por cada autor:

TextosporAutor (autor, cant_textos, fecha_ultima_public)

a) Analice cuál sería el recurso procedural más apropiado para completar la información de esta tabla a partir de los datos ya existentes en la/s tabla/s, incluya la implementación completa y plantee un ejemplo de su utilización.

b) Defina el/los trigger/s que estime necesarios para mantener la base consistente cuando se produzcan actualizaciones sobre Articulo.

Acciones Procedurales en Bases de Datos

Triggers, funciones y stored procedures

Ejercicio 8

Dadas la siguiente definición de tablas y disparadores T1, T2, T3 y T4 en PostgreSQL:

```
CREATE TABLE log_trigger (n SERIAL PRIMARY KEY, marca VARCHAR(50));

CREATE TABLE area_conocimiento (area varchar(5) primary key, nombre
varchar(5), descripcion varchar(20), ubicación text, tipo_area varchar(5));

-- T1
CREATE FUNCTION fn_bs() RETURNS trigger AS $$
BEGIN
  INSERT INTO log_trigger(marca) VALUES ('BEFORE STMT');
  RETURN NULL;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER t1 BEFORE UPDATE ON area_conocimiento
  FOR EACH STATEMENT EXECUTE FUNCTION fn_bs();

-- T2
CREATE FUNCTION fn_br() RETURNS trigger AS $$
BEGIN
  INSERT INTO log_trigger(marca) VALUES ('BEFORE ROW area=' || OLD.area);
  IF NEW.tipo_area NOT IN ('D','I','M') THEN
    NEW.tipo_area := 'M';
  END IF;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER t2 BEFORE UPDATE ON area_conocimiento
  FOR EACH ROW EXECUTE FUNCTION fn_br();

-- T3
CREATE FUNCTION fn_ar() RETURNS trigger AS $$
BEGIN
  INSERT INTO log_trigger(marca) VALUES ('AFTER ROW area=' || OLD.area);
  RETURN NULL;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER t3 AFTER UPDATE ON area_conocimiento
  FOR EACH ROW EXECUTE FUNCTION fn_ar();

-- T4
CREATE FUNCTION fn_as() RETURNS trigger AS $$
BEGIN
  INSERT INTO log_trigger(marca) VALUES ('AFTER STMT');
  RETURN NULL;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER t4 AFTER UPDATE ON area_conocimiento
  FOR EACH STATEMENT EXECUTE FUNCTION fn_as();
```

Si la tabla log_trigger está vacía y el estado inicial de la tabla area_conocimiento es el siguiente:

```
INSERT INTO area_conocimiento (area, nombre, descripcion, ubicación, tipo_area)
VALUES ('A', 'A1', 'a1', null, 'D'), ('B', '1', '100', null, 'D'), ('C', 'C1',
'45', null, 'M');
```

Acciones Procedurales en Bases de Datos

Triggers, funciones y stored procedures

y se ejecutan las siguientes tres sentencias S1, S2, S3, en este orden:

```
UPDATE area_conocimiento SET tipo_area = 'M' WHERE area = 'A'; -- S1
UPDATE area_conocimiento SET tipo_area = 'M' WHERE ubicacion IS NULL; -- S2
UPDATE area_conocimiento SET tipo_area = 'M' WHERE area = 'Z'; -- S3
```

Responda, justificando de manera clara y concisa en cada caso:

a) Cuál es la funcionalidad implementada mediante los cuatro disparadores, indicando el propósito de cada uno.
b) Para cada sentencia (S1, S2, S3) indique: qué disparadores se ejecutan y cuántas veces y qué filas se insertan en log_trigger y en qué orden.
c) Muestre el contenido final de AREA_CONOCIMIENTO y log_trigger.
d) Si la línea que ajusta el tipo de área a 'M' se trasladara del trigger T2 al trigger T3, sin modificar nada más, ¿qué efecto tendría sobre el resultado?
e) ¿Podría el trigger T1 registrar el identificador del área de conocimiento afectada, por ejemplo mediante la expresión 'BEFORE area =' || NEW.area, en lugar del texto fijo?